CASE STUDY 2: MODELS INTEGRATION

Course: CMU-CS 445 – *System Integration Practices*
Semester: 2
Academic: 2024–2025
Instructor: MSc. Nguyen Dang Quang Huy

Objective

After analyzing business and system issues in **Case Study 1**, students will now design an **integration model** between the two existing systems: *HUMAN (SQL Server)* and *PAYROLL (MySQL)*.

The purpose of this activity is to:

- Build a model that ensures consistent and synchronized data exchange.
- Define required APIs and data mappings for integration.
- Design the front-end structure for a dashboard interface that visualizes and manages the integrated data.

Project Scenario

Company X is ready to proceed with the integration of its HR and Payroll systems. The project requires defining clear integration logic, creating API specifications, and designing dashboard screens that will display and manage synchronized data.

The integration model should support the following key operations:

- Data exchange between SQL Server and MySQL databases.
- APIs for retrieving and updating HR, payroll, and attendance information.
- A dashboard interface to visualize HR and payroll metrics, employee data, and alerts.

Tasks and Requirements

Each student group must complete the following tasks and prepare the required design documents.

1. Software Requirement Specification (SRS)

Define all functional features that the integrated system should support.

Your SRS must include:

- **Employee Management:** View, search, add, update, delete employee records.

- **Payroll Management:** View and update salary information, view salary history.
- **Attendance Management:** Display attendance, leave days, and working status.
- **Reporting and Alerts:** Generate employee and payroll reports; create alerts for work anniversaries, leave violations, and payroll discrepancies.
- **Security:** User authentication and role-based access control (Admin, HR Manager, Payroll Manager, Employee).

2. UI/UX Design Document

Design the visual layout of the integrated dashboard (mock-up only).

Include the following screens:

- Dashboard Overview (summary of key HR and payroll metrics).
- Employee List with search and detail view.
- Payroll and Salary History view.
- Attendance Tracking view.
- Reports and Charts (employee statistics, payroll summaries).
- Alert and Notification Center.

Note: Use static mock data only. The design should be ready for API integration in Case Study 3.

3. Data Mapping Document

Define how data fields correspond between the two systems.

Your mapping must include:

- **Employees:** EmployeeID, DepartmentID, JobTitle, EmploymentStatus.
- **Departments:** Department name, code, and manager linkage.
- **Payroll:** Salary, bonus, deductions, and attendance.

Present your mappings clearly in a tabular format showing equivalent fields across both databases (*HUMAN→ PAYROLL*).

4. API Integration Plan

Design a set of RESTful API endpoints that will connect both systems (specification only).

Required Endpoints:

Method	Endpoint	Description
GET	/employees	Retrieve employee list from HUMAN
GET	/payroll	Retrieve payroll data from PAYROLL
POST	/add-employee	Add new employee and synchronize with PAYROLL
PUT	/update-employee	Update employee details in both systems
DELETE	/delete-employee	Delete employee (if no payroll dependencies)
GET	/attendance	Retrieve attendance data
GET	/reports	Retrieve HR and payroll reports
POST	/alerts	Send system alerts and notifications

5. Dashboard Mock-Up Requirements

Develop a **front-end prototype** using **HTML, CSS, JavaScript**, or frameworks like **React** or **ASP.NET MVC**.

Functional Requirements (static prototype only):

- Display lists, search functions, and tables using sample data.
- Design report views and charts using any library (Chart.js, Recharts, etc.).
- Display mock alerts and notifications.
- Prepare folder structure and UI components for future API integration.

6. Expected Deliverables

6.1 Document Deliverables

Document Name	Action (New / Update)	Detailed Description / Student Instructions
4. Software Requirement Specification (SRS) – Version 1	**New**	Translate the BRD into technical form. Write detailed functional requirements (Employee management, Payroll management, Reports, Alerts) and non-functional ones (security, performance, scalability). Describe the overall system architecture

		(frontend + backend + databases). Define key API endpoints to be implemented in later Case Studies.
5. Database Design and Data Mapping Document	**New**	Describe your understanding of both legacy databases (HUMAN) and PAYROLL). For each table, write its structure (fields, keys, relationships, description). Then create a data mapping table showing how fields correspond between the two databases (e.g., EmployeeID in HUMAN ↔ EmpNo in PAYROLL). Explain how each mapping will be used in integration.
6. UI/UX Design Document	**New**	Design the Dashboard layout using tools like Figma, Draw.io, or basic HTML/CSS. Create mockups for screens such as Employee List, Payroll Overview, Attendance, and Reports. Use static sample data only (no real database connection yet). Each screen must have a short explanation of its purpose and expected user actions.
7. Updated Project Proposal Document	**Update from Case 1**	Add a short section describing how integration and Dashboard development will be implemented. Update the timeline and technology choices if needed (for example, switching from JavaScript to React).

6.2 Software Functional Deliverables

Software Functions to Develop	Description / Implementation Guidance
1. Dashboard Mockup (Static)	Develop a **static Dashboard interface** using HTML/CSS/JS, React, or ASP.NET MVC. No real data

	connection yet. The Dashboard should display placeholders or mock data for: • Total Employees • Total Payroll Cost • Work Anniversary Reminders • Leave Alerts.
2. Employee Management (UI only)	Create screens for: • Viewing employee list (from HUMAN mock data) • Searching by name, department, job title • Adding/Editing employee (form layout only, no real save).
3. Payroll Management (UI only)	Build a UI to show: • Monthly salary overview (from mock data) • Search/filter by month and department.
4. Attendance Management (UI only)	Design layout to show attendance summary: Working days, leave days, absent days.
5. Reports & Alerts (UI only)	Prepare report templates: • Employee by department • Payroll summary • Alert list (work anniversary, leave violations).

7. Evaluation Criteria

Your group's submission will be assessed based on the following:

Criteria	Description
Integration Model Design	Logical design ensuring consistent data exchange.
Data Mapping Accuracy	Correct correspondence between HR and Payroll data fields.
API Definition Quality	Clarity and consistency of endpoint specifications.
Dashboard Design	Usability, structure, and readiness for future integration.
Documentation Quality	Completeness, clarity, and organization of all deliverables.